

Multi-dimensional Arrays and Basics of Pointers

Two-Dimensional Arrays

1. A two-dimensional array is specified using two subscripts where one subscript denotes row and the other denotes column.
2. C++ considers 2-D array as an array of 1-D arrays.
3. For example, a two-dimensional array consists of a certain number of rows and columns:

```
const int NUMROWS = 3;  
const int NUMCOLS = 7;  
int Array[NUMROWS][NUMCOLS];
```

	0	1	2	3	4	5	6
0	4	18	9	3	-4	6	0
1	12	45	74	15	0	98	0
2	84	87	75	67	81	85	79

Array[2][5] 3rd value in 6th column
Array[0][4] 1st value in 5th column

The declaration must specify the number of rows and the number of columns, and both must be constants.

Declaration of two dimensional Array

Two dimensional arrays must be declared before being used.

```
data_type    array_name [row_size][column_size];
```

for example if we want to store the marks obtained by 3 students in 5 different subjects , then we can declare a two dimensional array as

```
int marks[3][5];
```

Address of a 2-Dimensional Array

Computer stores the address of base element of array and the address of other elements is calculated using the following formula:

$$\text{Address (A[i][j])} = \text{base_address} + w\{M(J-1) + (I-1)\}$$

Example: Consider a 20×5 , 2-D array which has base address = 1000 and number of words per memory location of the array = 2. now compute the address of element Marks[18,4] assuming that elements are stored in row major form.

Initialization of 2-D Array

2D array is initialized in the same way as a 1D array, For example:

```
int marks[2][3] = {90,87,78,68,62,61};
```

The initialization of 2 D array is done row by row , this statement can also be written as :

```
int marks[2][3] = { {90,87,78}, {68,62,61} };
```

In order to initialize the entire 2D array to zero, simply specify the first value as zero, i.e. , simply write

```
int marks[2][3] = {0};
```

If some values are missing in the initializer then it is automatically set to zero.

```
int marks[2][3] = { {90,87,78} };
```

Accessing the Elements

The elements in a multidimensional array are stored in contiguous memory locations. While accessing the elements, remember that the last subscript varies most rapidly whereas the first subscript varies least rapidly.

Individual elements of a 2D array can be initialized using assignment operator as shown below:

`marks[1][2] = 79; or mark[1][2]=marks[1][1] + 10;`

In order to input the values from the keyboard, we must use the following code:

```
for (int row = 0; row < 2; row++) {  
    for (int col = 0; col < 3; col++) {  
        cout << Array1[row][col];  
    }  
    cout << endl;  
}
```

WAP a program to print elements of a 2D Array

```
#include<iostream>
using namespace std;
int main()
{
    int marks[2][5];
    cout << "Enter the marks for marks of 2 students in 5 subjects:" << endl;
    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 5; j++)
        {
            cin >> marks[i][j];
        }
    }
    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 5; j++)
        {
            cout << "marks of [ " << i << "students in subject [ " << j << " ] = " << marks[i][j] << endl;
        }
        cout << endl;
    }
    system("pause");
    return 0;
}
```

Operations on 2D Arrays

Following operations can be performed on a 2D array:

Transpose: Transpose of a $m \times n$ matrix A is given as a $n \times m$ matrix B where, $B_{i,j} = A_{j,i}$.

Sum: Two matrix are compatible with each other can be added together thereby storing the result in the third matrix. Two matrix are said to be compatible when they have same number of rows and same number of columns. Elements of the matrix can be added by writing: $C_{i,j} = A_{i,j} + B_{i,j}$

Difference: Two matrix are compatible with each other can be subtracted thereby storing the result in the third matrix. Two matrix are said to be compatible when they have same number of rows and same number of columns. Elements of the matrix can be subtracted by writing: $C_{i,j} = A_{i,j} - B_{i,j}$

Product: two matrix can be multiplied with each other if the number of columns in the first matrix is equal to the first matrix is equal to the number of rows in the second matrix. Therefore, $m \times n$ matrix A can be multiplied with a $p \times q$ matrix if $n = q$. Elements of the matrices can be multiplied by writing :

$$C_{i,j} = \sum A_{i,k} B_{i,k} \text{ for } k=1 \text{ to } k < n$$

2D Array for Inter-Function Communication

There are three ways of using 2-D arrays to function:

1. Passing individual elements
2. Passing a row
3. Passing an entire 2D array.

Passing a row

1. A row of 2D array can be passed by indexing the array name with the row number .
2. when a single row of 2D array is send, then called function receives a 1 D array.

```
#include<iostream>
using namespace std;
void func(int arr[])
{
    int i;
    for (i = 0; i < 3; i++)
    {
        cout << arr[i] * 10<<endl;
    }
}
int main()
{
    int arr[2][3] = { { 1, 2, 3 }, { 4, 5, 6 } };
    func(arr[0]);
    system("pause");
    return 0;
}
```

Passing the Entire Array

1. To pass a 2D array to a function, we use the array name as the actual parameter.
2. However the parameter in the called function must indicate that the array has two dimensions.

```
#include<iostream>
using namespace std;
void func(int arr[2][3])
{
    int i,j;
    for (i = 0; i < 2; i++)
    {
        for (j = 0; j < 3;j++)
            cout << arr[i][j] * 10<<endl;
    }
}
int main()
{
    int arr[2][3] = { { 1, 2, 3 }, { 4, 5, 6 } };
    func(arr);
    system("pause");
    return 0;
}
```

Multidimensional Array

1. Multi dimensional array is said to be array to arrays.
2. Like we have one index in 1D array two indices in 2D array , in the same way we have n indices in an n dimensional array.
3. a multidimensional array can contain as many indices as needed and the requirement of the memory increases with the number of indices used.
4. However , we will hardly use more than three indices in any program.
5. A three dimensional array consists of pages. Each page in turn contains m rows and n columns.

```
#include<iostream>
using namespace std;
int main()
{
    int arr[2][3][5] = { { 1, 2, 3 }, { 4, 5, 6 } };
    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            for (int k = 0; k < 5; k++)
            {
                cout << arr[i][j][k];
            }
            cout << endl;
        }
        cout << endl;
    }
    system("pause");
    return 0;
}
```

Introduction to Pointers

Consider the declaration

```
int x = 3;
```

This declaration tells the compiler to:

1. Reserve space in memory to hold the integer value.
2. Associate the name `x` with this memory location.
3. Stores the value 3 at this location.

Pointers are frequently used in C++ language as they have a number of useful applications. These applications include the following:

1. To pass information back and forth between function and its reference point.
2. Enable programmers to return multiple data items from a function via function argument.
3. Provide an alternate way to access individual elements of array.
4. To pass arrays and strings as function arguments.
5. To create complex data structures such as trees , linked list ,linked stack, linked queue, and graphs.
6. For dynamic memory allocation of a variable.

Dynamic Pointer Variable

A pointer provides access to a variable by using the address of that variable.

A pointer variable is therefore , a variable that stores the address of another variable.

The general syntax of declaring pointer variable can be given as follows:

```
Data_type *ptr_name;
```

Here data_type is the data type of the value that the pointer will point to .
For example,

```
int *pnum;
```

```
char *pch;
```

```
float *pfnum;
```

In each of the above statements, a pointer variable is declared to point to a variable of the specified data type.

Although all these pointers , pnum, pch, and pfnum point to different data types but they will occupy the same amount of space in memory .

```
#include<iostream>
using namespace std;
int main()
{
    int *pnum;
    char *pch;
    float *pfnum;
    double *pdnum;
    long *plnum;
    cout << sizeof(pnum) << endl;
    cout << sizeof(pch) << endl;
    cout << sizeof(pfnum) << endl;
    cout << sizeof(pdnum) << endl;
    cout << sizeof(plnum) << endl;
    system("pause");
    return 0;
}
```

Now let us declare an integer pointer variable:

```
int x = 10;  
int *ptr;  
ptr = &x;
```

In the above statement ptr is the name of pointer variable . The ‘*’ informs the compiler that ptr is a pointer variable and int specifies that it will store the address of an integer variable. In last statement ptr is assigned the address of the & operator retrieves the lvalue(address) of x, and copies that to the contents of the pointer ptr.

We can dereference a pointer , i.e. refer to the value of the variable to which it points, by using unary ‘*’ operator . Therefore * is equivalent to writing the value at the address.

```
#include<iostream>
using namespace std;
int main()
{
    int num, *pnum;
    pnum = &num;
    cout << "Enter the number:" << endl;
    cin >> num;
    cout << num << endl;;
    cout << "the num that was entered " << *pnum << endl;
    cout << "the address of the number in memory is " << &num << endl;
    cout << pnum;
    system("pause");
    return 0;
}
```

One thing to be remember is that the data type of the pointer variable and the variable whose address it will store must both be of the same type.

Therefore the following code is not a valid code:

```
int x = 10;
```

```
float y = 2.0;
```

```
int *Px ;
```

```
float *Py;
```

```
Px= &y;
```

```
Py = &x;
```

Note that it is not necessary that the pointer variable will point to the same variable throughout the program. It can point to any variable as long as data type of the pointer variable is same as that of the variable it points to.

```
#include<iostream>
using namespace std;
int main()
{
    int a=3,b=5, *pnum;
    pnum = &a;
    cout << *pnum << endl;
    cout << pnum<<endl;
    pnum = &b;
    cout << *pnum << endl;
    cout << pnum <<endl;
    system("pause");
    return 0;
}
```